

Improving the matrix multiplication exponent with a MacBook and a proof assistant

A CN-certified level-four bound, conditional on the combination-loss theorem

BSD (bsd.developer@proton.me)
Independent Researcher

4 October 2026

Abstract

We give an exact certificate for the rational-field matrix multiplication exponent bound $\omega_{\mathbb{Q}} \leq 43740354192942056903/2^{64} = 2.371169352063661 \dots$, below the displayed bound 2.371177 of Dupont et al. The bound is the conclusion of a Lean theorem about the exact 459 MB certificate, checked under the CN (native-certified) profile: a general Lean soundness proof plus one certificate-specific native-evaluation axiom. A full Lean CN evaluation and a whole-module replay accepted the certificate, an independent exact Rust checker agreed, and compiled statements, axiom dependencies and deterministic replay artifacts were checked. The exponent inference uses the combination-loss feasibility theorem of Alman et al., as stated by Dupont et al., represented by an unproved Lean axiom. Native evaluation and its compiler/runtime dependencies remain explicit. Readers can recheck the claim from the release package without the private numerical optimizer; a clean-room reader run from the published release assets passed (Appendix C). Search and certification ran on a MacBook Pro (Apple M1 Max, 64 GiB unified memory) using CPU computation. The search builds small second-order tangent models and restores weighted branch balance before ranking candidates by the original nonsmooth objective. In eight recorded same-origin width comparisons, the wider nested model always scored lower, as its construction makes likely; only four met the recorded marginal compute-efficiency threshold. Earlier matched controls in the same campaign favored plain Adam. This is descriptive evidence from one adaptive campaign, not a claim of optimizer superiority.

1. Result and scope

Every recent improvement to ω uses the laser method; the current refinement is combination-loss analysis (Duan, Wu, and Zhou 2023; Vassilevska Williams et al. 2024; Alman et al. 2025), building on earlier laser-method bounds (Coppersmith and Winograd 1990; Le Gall 2014; Alman and Vassilevska Williams 2021). Alman et al. formulated the finite optimization problem at the core of this approach, showed that any feasible solution yields an upper bound on ω , and solved it at recursion level $\ell^* = 3$ with sequential quadratic programming (Alman et al. 2025; Gill, Murray, and Saunders 2002). Dupont et al. restated the problem (their Eq. (11)) and its feasibility theorem (their Theorem 1, attributed to Alman et al.), solved it at $\ell^* = 4$ with a gradient method and AlphaEvolve, and reported $\omega < 2.371177$ (Dupont et al. 2026). We use that level-four formulation with Coppersmith–Winograd parameter $q = 5$ and $\ell^* = 4$, conditional on their feasibility theorem (§3). The contribution here is a further numerical refinement with an exact, independently checkable certificate and an explicit formal trust boundary. The same checking pipeline previously certified a level-three bound, $\omega \leq 2.371281376 \dots$, below Alman et al.’s level-three value (BSD 2026); the present paper applies it at level four.

Let $\omega_{\mathbb{Q}}$ denote the rational-field matrix multiplication exponent defined in the supplied Lean source. The result is

$$\omega_{\mathbb{Q}} \leq \Omega_* := \frac{43740354192942056903}{18446744073709551616} = 2.3711693520636610535 \dots \quad (1)$$

Equation (1) is the Lean theorem’s conclusion under the CN and combination-loss assumptions specified in §3. It is not an axiom-free derivation of the exponent bound. The exact rational is authoritative; the displayed decimals are approximations. We make no all-fields claim.

The certified value is approximately 7.64794×10^{-6} below Dupont et al.’s displayed 2.371177. That comparison uses their displayed upper bound. Their unrounded value and their verification code had not been released when this paper was written, so no claim is made about their unpublished checkpoint.

Only the final search043 endpoint is certified; earlier milestones of the same campaign are numerical observations (§6).

Sections 2–4 describe the certificate, the trust boundary and reader verification. Sections 5–6 describe how the point was found, §7 discusses limitations, and §8 outlines where a further improvement could come from.

2. Exact certification

The selected physical point was rationalized and supplied with fresh maximum-entropy witnesses and directed logarithm enclosures. This follows the certification scheme of Dupont et al.:

- their Lemma 1 (§2.5): a strictly positive witness y with the target marginals and Lagrange residual ε certifies $H(y) \leq H_D^{\max}(\rho) \leq H(y) + 2\varepsilon$;
- their §4: exact rational evaluation with outward-rounded logarithms.

Our implementation proves the Lemma 1 bound and the directed checker sound in Lean. Exact checking verifies the domain conditions and the sufficient feasibility inequality at (1). The Rust and Lean implementations do not use the numerical optimizer’s floating-point score as evidence of feasibility.

Table 1: The selected point’s numerical score and its distinct outward-certified claim (1).

Quantity	Value
Numerical search score	2.3711693500058737
Exact certified bound, approximately	2.3711693520636611

The certified claim exceeds the numerical score by approximately 2.05779×10^{-9} . This difference includes rationalization and outward certification bounds. It is not a floating-point error estimate.

The canonical certificate contains **459,013,469 bytes**, with SHA-256

1a05ebc4e32c1a2121e6da2020c4321cc1d505ea2564b29e3d64eb12ee5ff521

The generated module is `MatrixMath.Generated.0mega_1a05ebc4e32c1a21`. Its acceptance theorem applies to the complete literal bytes, claim and digest. Its exponent theorem concludes (1). The exact declaration names, elaborated statement hashes and transitive axioms are supplied in `compiled-assurance.json`.

Two full CN executions completed on the same machine and pinned toolchain:

- the first evaluation; and
- a replay after retrieval through an initially empty local content-addressed store.

The replay regenerated and evaluated the whole module; it did not reuse the first acceptance Boolean or theorem as its computation. It confirms determinism and lossless transport, not independence from the toolchain. Exact Rust checks, compiled theorem/axiom audits, complete trust ledgers and all four deterministic artifact comparisons also passed.

3. Formal trust boundary

The claim (1) is the conclusion of a Lean theorem about the exact certificate bytes. Table 2 summarizes what that theorem rests on.

Table 2: What is proved, what is assumed, and what corroborates it.

Machine-checked	Certificate acceptance (one full CN evaluation, a whole-module replay, and a clean-room reader run from the release assets, Appendix C), soundness of the directed checker, and the maximum-entropy bound
Assumed	Lean’s three standard axioms; one certificate-specific native-evaluation axiom, with its compiler/runtime/GMP trust; and AX1_combination_loss, the cited combination-loss theorem, including the fidelity of its Lean transcription
Corroborated	An independent exact Rust checker agrees

Beyond the soundness proofs of §2, which connect byte acceptance to the defined feasibility predicate in Lean (Moura and Ullrich 2021), the result theorem depends on three groups:

1. **Standard logical axioms:** `Classical.choice`, `propext` and `Quot.sound`.
2. **Certificate-specific native evaluation:** one `native_decide` axiom for the closed acceptance computation. This adds trust in the relevant pinned compiler, native runtime and big-integer implementation.
3. **The mathematical bridge:** `MatrixMath.AX1_combination_loss`, representing the combination-loss feasibility theorem (Alman et al. 2025) as stated in Dupont et al.’s Theorem 1 (Dupont et al. 2026). It connects the checked feasibility conditions to the exponent bound. Its proof is not formalized in this package. Alman et al. computed at $\ell^* = 3$; the level-four instance relies on the theorem as stated for general ℓ^* in Dupont et al.’s Theorem 1. We have not independently re-derived that theorem or the equivalence of the level-four reformulation; both are inherited through this axiom.

AX1 is stated for every $q \geq 1$ and $\ell^* \geq 2$. These restrictions are enforced by the well-formedness predicate inside `CombinationLossFeasible`. The axiom is only as trustworthy as the Lean transcription of Dupont et al.’s Eq. (11). If that transcription were not faithful, AX1 could be false, and the result theorem would not establish the bound. The fidelity of the definitions in `lean/MatrixMath/Spec/Instance.lean` to Eq. (11) is therefore part of what a reader must audit. `verification-source/docs/traceability.md` maps each spec equation to its Lean definition and Rust implementation for that audit.

Lean kernel soundness is a metatheoretic assumption, not a further Lean axiom. The result’s transitive dependency list is mechanically recorded; that transparency does not remove the assumptions themselves. Rust agreement is independent corroboration of the computation, not of the transcription, and not a replacement for the Lean theorem’s premises. The numerical optimizer, AI assistants and certificate producer remain outside the result’s logical authority.

Proving the bridge in Lean and replacing native evaluation by kernel-checkable certificate acceptance would strengthen the result. Neither has been completed for this released package. Work on those extensions does not retroactively change this theorem’s axiom list.

4. Independent reader verification

The matching package is `certifications/l4-2026-10-04-search043`. Its `verification-runbook.md` separates four distinct operations:

1. Check supplied sizes, hashes and lossless theorem transport with `python3 verify-files.py`.
2. Build the pinned Rust and Lean source in an isolated copy.
3. Run the independent exact Rust check on the complete certificate.
4. Run full CN certification, compare the generated module and compiled assurance, and inspect the new runtime trust ledger.

The last two commands, from the isolated `verification-source` directory, are

```
target/release/mm verify ../payloads/certificate.json --skip-lean --json
target/release/mm prove ../payloads/certificate.json --profile cn --json
```

Rust 1.94.0, Lean 4.33.0 and the supplied dependency locks fix the intended build environment. File hashing checks identity; compilation checks source buildability. Neither alone establishes fresh certificate acceptance.

The `verify-files.py` inventory is stored in the same package it checks. The external trust anchors are the certificate and module digests printed in §2 and Appendix A, together with the release tag.

Resources. Table 3 gives the recorded durations of the original checking commands, including their prerequisite work.

Table 3: Result-specific checking durations. The CN commands include Rust prerequisites, module generation, dependency work and compiled auditing; these are not isolated Lean kernel times. The two standalone Rust checks are separate commands, not durations subtracted from the CN commands.

Check	Command elapsed (s)
First exact Rust check	793.11
First full CN certification	13,908.37
Retrieved exact Rust check	827.38
Whole-module CN replay	14,984.84

Seven recorded sequential result-specific commands, including production, exact target selection and local retrieval, total about 9.13 hours. This excludes setup/builds, separate audits and reviews, and unknown-duration intervals. No whole-research compute total is asserted.

The recorded runs used a MacBook Pro (Apple M1 Max) with 64 GiB unified memory. Peak whole-system memory in use was about 34 GiB during the first CN run and 37 GiB during the replay. Both figures include roughly 20 GiB already in use by other processes when each run started. In the clean-room run the `mm`, `lake` and `lean` processes together peaked at 15.4 GiB resident (Appendix C). We therefore expect about 48 GiB of total RAM to suffice, but have not tested a smaller machine.

Clean-room reader run. On 4 October 2026 the documented reader path was executed end to end, exactly as a stranger would:

1. the release assets were downloaded fresh from the release page and checked against the release checksum file;
2. the package was unpacked and `verify-files.py` passed;
3. the reduced workspace was built from the pinned sources with freshly fetched dependencies;
4. the exact Rust check and the full CN certification ran.

Result: pass. The exact Rust check returned `VERIFIED` with output byte-identical to the runbook, and the full CN certification exited 0. The regenerated module was byte-identical to the shipped module and matched the published digest, and the emitted `compiled-assurance.json` (statements, statement digests and transitive axioms) was byte-identical to the supplied one. Durations and peak memory are in Appendix C.

This run establishes that the released package, not only the original full-source workspace, reproduces acceptance. Its source closure and runtime ledger are recorded separately from the original full-source records.

The certificate and compressed theorem are separate large payloads; GitHub’s automatic source archives do not contain them. The release identifier is `14-2026-10-04-search043`. The associated minimal verification archive, certificate, compressed theorem and download checksums provide the inputs listed in the runbook. The search program and its private checkpoints are not needed to reproduce certificate verification.

5. Objective and local search model

5.1 Feasibility and the hard score

A physical point p consists of distributions and leaf parameters on a recursion tree. Different paths retain different parameters even when their shapes coincide. Raw coordinates θ map to $p = \Phi(\theta)$ through softmax and interval parameterizations, following Dupont et al. (their §3.1). Maximum-entropy subproblems are solved by Sinkhorn scaling (Sinkhorn and Knopp 1967; Cuturi 2013), as there. The search is approximate; the certificate checks the resulting physical data exactly.

Write E for the retained exponent contribution and M for the matrix-size exponent. At target τ , the feasibility surplus is

$$F_\tau(\theta) = E(\theta) + \tau M(\theta) - C, \quad C = 8 \log_2 7. \quad (2)$$

When $M > 0$, $F_\tau \geq 0$ is equivalent to $(C - E)/M \leq \tau$. This is the standard parametric (Dinkelbach) treatment of a ratio objective (Dinkelbach 1967). The recorded continuation held τ fixed at 2.371170 rather than updating it, and ranked candidates by the true quotient (3). When $M(\theta) > 0$, the numerical objective is

$$\Omega_{\text{hard}}(\theta) = \max \left\{ 0, \frac{C - E(\theta)}{M(\theta)} \right\}. \quad (3)$$

The construction has twenty three-way minima over branches X, Y, Z : six root sites, twelve interior sites, one level-two retained-exponent site and one matrix-size site. Thus

$$F_\tau = g_0 + \sum_{i=1}^{20} a_i \min_k v_{i,k}, \quad a_i \geq 0. \quad (4)$$

The coefficients include root weights, which are themselves optimized, and, at the matrix-size site, the target τ . Branch equality and weighted branch balance therefore measure different things.

5.2 Tangent models and finite restoration

At a model origin θ_0 , choose one minimum-attaining reference branch at each site. The other two branches give forty normalized contrasts

$$c_{i,k}(\theta) = \frac{a_i(\theta)(v_{i,k}(\theta) - v_{i,k_i^*}(\theta))}{M_0}, \quad k \neq k_i^*, \quad (5)$$

with Jacobian $J = Dc(\theta_0)$. Here $M_0 = 6.051764847062773$ is a fixed normalization constant recorded with the search protocol, close to but not equal to $M(\theta_0)$ (for example, $M = 6.0530$ at the final origin). Rank-checked solves with $JJ^\top$ project directions into the approximate tangent space $\ker J$. This is a local numerical construction; it does not prove that a global smooth tie manifold exists throughout the search.

The branch weights are fitted at the origin as the least-squares multipliers

$$\lambda = \arg \min_{\lambda} \|\nabla f_{\text{ref}} - J^\top \lambda\|, \quad (6)$$

where f_{ref} is the loss with each minimum replaced by its reference branch. The two fitted multipliers at site i are the weights of its non-reference branches; the reference branch receives the remainder. A model is admitted only if every site's weights are nonnegative and sum to one, so that they form a Clarke subgradient of the hard loss (Clarke 1983).

A model fits the normalized smooth branch-mixture loss with these fixed weights on a small span $U = [u_1, \dots, u_m]$:

$$q(z) = \gamma^\top z + \frac{1}{2} z^\top H z. \quad (7)$$

Reduced curvature comes from central differences of fresh first gradients (Nocedal and Wright 2006), checked at two step sizes and for mixed-product symmetry. These are numerical consistency tests, not interval bounds on the Hessian. The model suggests a displacement. From the seventeenth recorded stage onward, its largest raw-coordinate change was scaled down to at most 0.25; earlier stages instead ended the model when that change exceeded 0.5. Finite evaluations and corrections toward weighted balance follow; each correction is a minimum-norm step using the Jacobian frozen at the origin. Candidates are ranked by (3), not by the quadratic prediction.

The directions are built in order from successive projected residuals, so the span is nested and Krylov-like. A model of width m uses the first m directions of any wider model from the same origin. The step minimizes the quadratic on that span exactly and is then scaled, an approach related to subspace trust-region and truncated conjugate-gradient methods (Steihaug 1983; Nocedal and Wright 2006) but not a trust-region subproblem solve. This procedure starts with a small direction set and sometimes expands it to twelve or twenty directions. The recorded continuation also contains smaller retained models and partial expansions, including an eleven-direction comparison. Model width is a number of coupled directions in the full parameter space, not a search over six or twelve individual parameters.

5.3 Why correction matters

For convex branch weights w_i , define

$$s_i = \sum_k w_{i,k} v_{i,k} - \min_k v_{i,k} \geq 0, \quad S = \sum_i a_i s_i.$$

Replacing each minimum by its weighted average yields a mixture surplus F_{mix} with the exact accounting identity

$$F_\tau = F_{\text{mix}} - S, \quad \Delta F_\tau = \Delta F_{\text{mix}} - \Delta S. \quad (8)$$

A smooth proposal can gain mixture surplus while losing hard surplus through branch slack. Restoration addresses that finite penalty. Equation (8) is elementary algebra; it is not a new theorem establishing optimizer convergence.

The overall scheme belongs to established families:

- separating directions along an active set from directions transverse to it, as in partial smoothness and $\mathcal{VU}$ methods (Lewis 2002; Lemaréchal, Oustry, and Sagastizábal 2000; Mifflin and Sagastizábal 2005);
- correcting a tangent step back to the constraint set, as in second-order corrections for sequential quadratic programming (Nocedal and Wright 2006).

Those theories do not automatically give convergence guarantees for this nonconvex program, and we claim none.

6. Observed search

The search used CPU float64 on a MacBook Pro (Apple M1 Max) with 64 GiB unified memory. No GPU or cloud computation was used for the numerical search or the recorded local certificate executions. The hardware matters mainly through memory capacity. A laptop with 64 GiB of unified memory could hold the full level-four search state (about 4.66 million free coordinates, with dozens of full-length gradient arrays per model), and it could hold the Lean evaluation of a 459 MB generated theorem. We show feasibility on this machine, not that Apple hardware is uniquely required; any machine with comparable memory should work.

AI systems assisted with engineering, analysis and proof development under the author’s direction. They were not the numerical optimizer: programs generated and evaluated finite candidates, which were checked independently before numerical promotion. The aggregate plotting data supplied with this paper allow its descriptive charts to be examined, but do not reproduce the private optimization campaign. Statements below about the exploratory phase rest on the author’s records.

Exploratory phase. The campaign began with randomly initialized symmetric level-four search and a lift of a level-three solution. It then opened path-dependent parameters, tested fixed-block probability models, and added the branch-aware corrections of §5. This phase ended at the numerical milestone 2.371174031813689.

Negative results from that phase. Several mathematically motivated variants lost to ordinary optimization:

- In a matched-time control from a common checkpoint, 25 recomputed branch corrections gained 1.38×10^{-6} in 4546 s. Fresh Adam (Kingma and Ba 2015) gained 7.70×10^{-6} in 4552 s.
- Repeated reduced-Newton composition gained 1.52×10^{-6} in 2375 s. The first Adam checkpoint at a matched or later time had gained 4.37×10^{-6} in 2637 s.
- In a later equal-allowance comparison, both arms timed out. Adam’s completed checkpoints did not improve its baseline, while a saved adaptive trial was later validated.

These comparisons do not rank optimizers in general. They show that the continuation’s method is one workable choice in this basin, not a demonstrated improvement over tuned first-order search.

Continuation. The aggregate data accompanying this paper cover 41 recorded continuation stages with global-gain accounting, including one rejected trial. Figure 1 shows the accepted numerical trajectory and selected endpoints. These are sequential observations from one adaptive campaign, not independent trials. Only the selected final endpoint received the exact certificate (1).

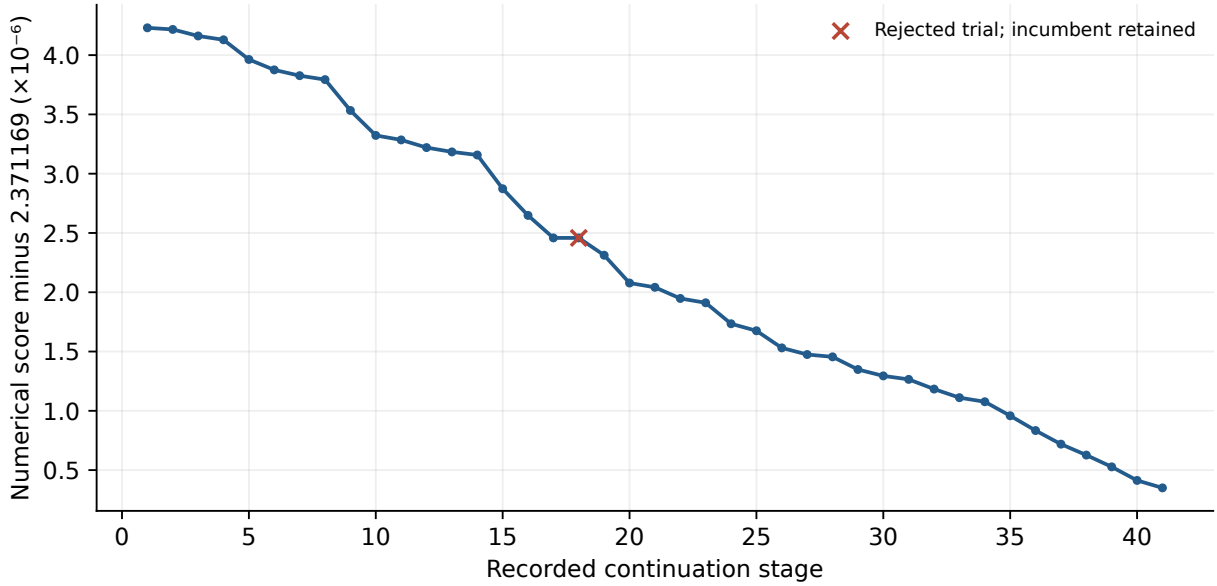


Figure 1: Accepted numerical scores across 41 recorded continuation stages. The horizontal axis is record order, not elapsed time. Rejected stages retain the preceding incumbent. The plotted numerical endpoint is distinct from the certified claim (1).

Table 4: A subset of recorded numerical incumbents. Full plotted data are in `data/search-progress.csv`. The first plotted stage is already below the exploratory-phase milestone; the graph does not represent every intervening preparation step.

Selected continuation point	Numerical score	Model width
First plotted stage	2.3711732303548674	20
Later matched comparison	2.3711710779474715	12
Subsequent matched comparison	2.3711707343084147	12
Partial expansion	2.3711699581586510	11
Later twelve-direction step	2.3711698336026850	12
Penultimate point	2.3711694128460494	6
Selected search043 endpoint	2.3711693500058737	6

6.1 Wider models and the cost of extra directions

Eight recorded comparisons have both a wider score and a marginal funding calculation. Seven compare six with twelve directions; one compares six with eleven. In each, the wider model found a lower recorded hard score from the same origin. This is largely expected by construction. The narrower model’s directions are a prefix of the wider model’s nested sequence. So whenever the reduced Hessian is positive definite, the wider quadratic step predicts at

least as much gain. Only displacement capping and finite restoration can reverse the order. The informative quantity is therefore cost.

A comparison was **funded** when three conditions held:

1. the extra global gain exceeded $\max(10^{-9}, 10u)$, where u is the conservative readout uncertainty;
2. the extra gain per charged hour exceeded a rate floor ρ , preregistered before the comparison from the campaign's recently realized global gain per recorded phase hour; and
3. both widths met weighted balance $\leq 10^{-11}$.

Charged time is the stage's phase time minus the six-direction model span and the six-direction trials. Shared or unassigned computation is therefore charged to the wider model. Four comparisons were funded and four were not (Figure 2). Two further attempted comparisons lack the required paired data and are excluded from that count.

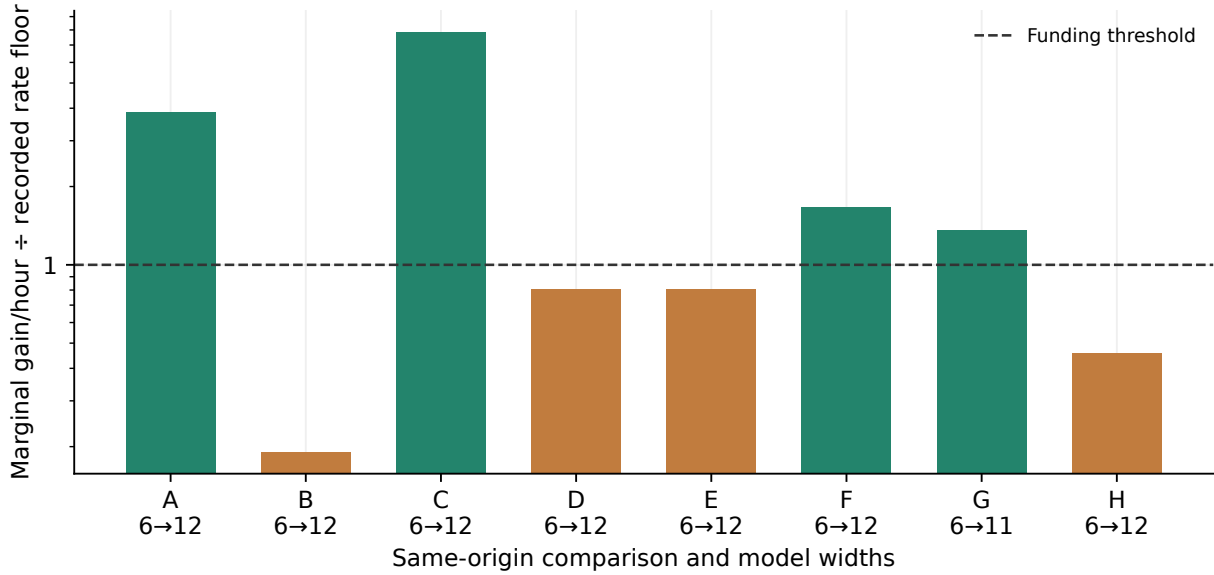


Figure 2: Marginal wider-model gain per recorded computation hour divided by the preregistered rate floor. A ratio of one is the funding threshold. All eight wider scores improved on their matched narrow scores; only four passed this economic threshold. Comparison G has eleven directions; the others have twelve.

The floor moves with the campaign's own recent progress. It is not a statistical significance test or a fixed mathematical property of a width. Balance qualification and numerical promotion are separate gates; a favorable rate does not establish an admissible candidate. The defensible observation is that extra directions did not consistently justify their additional cost, even when they lowered the score. The data do not show that six directions generally beat twelve or twenty, and do not establish a universal best expansion schedule.

6.2 Smooth gain versus the original objective

Comparison E illustrates the point:

Quantity at the origin of comparison E (twelve directions)	Normalized gain
Raw proposal, mixture gain	2.33451×10^{-7}
Raw proposal, hard gain	-7.98209×10^{-6}
After restoration, hard gain	2.34389×10^{-7}

These are changes in surplus divided by the fixed origin normalization, not differences of numerical exponent scores. The contrast between the raw and restored outcomes illustrates (8): an accurate prediction for a smooth mixture alone is insufficient.

The supplied `data/model-predictions.csv` distinguishes raw mixture gain, raw hard gain and corrected hard gain for eleven observed proposals. Corrected steps differ from the original model displacement, so their outcomes are not direct measures of quadratic prediction accuracy.

6.3 Geometry of the selected endpoint

Three recorded readouts of the final point agree at 2.3711693500058737 . They use indexed, reference and serialized/reloaded indexed evaluations. Each reports maximum absolute normalized weighted contrast 2.26042×10^{-13} . The largest *unweighted* branch spread is instead 1.27715×10^{-7} , at a root site with small weight. Root weights range from approximately 9.97040×10^{-7} to 0.436922 .

Thus the endpoint is closely balanced in the weighted objective, while not all unweighted branch triples agree to the same tolerance. Branch balance is a search heuristic only. The certificate checks the feasibility inequality, and ties play no role in its validity.

6.4 Compute accounting

The 41 recorded continuation stages charge approximately **18.82 hours** to newly recorded model, endpoint, reconciliation and metric-setup phases. They also charge **17.09 minutes** of recorded numerical-review time.

- These totals include the rejected stage.
- They exclude preparation and other unavailable intervals.
- Retained models may borrow earlier work.
- They are not total CPU hours or the wall-clock duration of the research project.

The exploratory phase’s optimizer records already contain approximately 483 process-elapsed hours, before its later derivative and certification stages. That separate partial accounting prevents interpreting the 18.82-hour continuation as the cost of finding the bound from scratch. The work demonstrates feasibility of local laptop refinement and checking; it does not establish a comparative speed advantage over GPU-based search.

7. Interpretation and limitations

The numerical gain comes from further refinement within the existing level-four formulation. It is not a new asymptotic matrix-multiplication construction or a resolution of known barriers (Ambainis, Filmus, and Le Gall 2015; Alman and Vassilevska Williams 2023). The methodological observation is narrower: small tangent models, finite restoration and adaptive investment in extra directions produced useful progress in the observed basin. Better smooth predictions did not remove the need to test the nonsmooth hard objective.

The campaign is adaptive and its origins are sequentially related. The record does not include:

- a randomized multi-seed comparison;
- a proof of local or global optimality;
- a uniform curvature bound or convergence guarantee;
- a theorem explaining the near-uniform branch multipliers (Appendix D shows that, at almost every site, the data cannot distinguish them from $1/3$).

The selected endpoint’s certificate does not validate a causal account of how it was found.

8. Outlook: where a further improvement could come from

A retrospective analysis of the saved search evidence (Appendices D and E) locates the remaining slack in this formulation. It is numerical analysis of one optimum, not a bound.

1. **The combination-loss penalty is not the bottleneck.** At the search043 optimum the maximum-entropy penalty $P = H^{\max} - H$ is essentially zero. Removing it entirely would lower ω by at most about 1.6×10^{-6} to first order.
2. **The conditioning terms carry the measurable slack.** The η terms charge shapes grouped by one coordinate the entropy of the group’s mixture. Full per-shape conditioning would be worth about 7.3×10^{-4} to first order. After re-optimization at level three, an idealized relaxation is worth about 2.4×10^{-3} . These are relaxations, not bounds.

3. **That slack is not reachable by refining compatibility inside the current argument.** Making “compatible” mean “can be contained in the triple”, which is the most an admissible definition can do, recovers:

- exactly 0 at level three;
- at most 6.9×10^{-10} at level four;
- at most 1.4×10^{-9} at the root, against a root gap of 2.0×10^{-4} .

The natural grade-1 refinement was already rejected by Duan, Wu and Zhou (Duan, Wu, and Zhou 2023), and our counts confirm it.

Reaching the conditioning slack therefore appears to need a mechanism beyond variable zero-outs, one that controls how blocks are coupled on shared positions. Alman et al. (Alman et al. 2025) anticipate this when they say “a truly new idea is needed”.

Bibliography

- Alman, Josh, Ran Duan, Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. 2025. “More Asymmetry Yields Faster Matrix Multiplication.” In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2005–39. SIAM. <https://doi.org/10.1137/1.9781611978322.63>.
- Alman, Josh, and Virginia Vassilevska Williams. 2021. “A Refined Laser Method and Faster Matrix Multiplication.” In *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 522–39. SIAM. <https://doi.org/10.1137/1.9781611976465.32>.
- . 2023. “Limits on All Known (and Some Unknown) Approaches to Matrix Multiplication.” *SIAM Journal on Computing* 52 (6): FOCS18-285-FOCS18-315. <https://doi.org/10.1137/19M124695X>.
- Ambainis, Andris, Yuval Filmus, and François Le Gall. 2015. “Fast Matrix Multiplication: Limitations of the Coppersmith–Winograd Method.” In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing (STOC 2015)*, 585–93. ACM. <https://doi.org/10.1145/2746539.2746554>.
- BSD. 2026. “Improving a Matrix Multiplication Bound with a MacBook and a Proof Assistant.” Zenodo. <https://doi.org/10.5281/zenodo.22101463>.
- Clarke, Frank H. 1983. *Optimization and Nonsmooth Analysis*. Canadian Mathematical Society Series of Monographs and Advanced Texts. New York: Wiley.
- Coppersmith, Don, and Shmuel Winograd. 1990. “Matrix Multiplication via Arithmetic Progressions.” *Journal of Symbolic Computation* 9 (3): 251–80.
- Cuturi, Marco. 2013. “Sinkhorn Distances: Lightspeed Computation of Optimal Transport.” In *Advances in Neural Information Processing Systems 26 (NIPS 2013)*, edited by C. J. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Weinberger, 26:2292–2300. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2013/file/af21d0c97db2e27e13572cbf59eb343d-Paper.pdf.
- Dinkelbach, Werner. 1967. “On Nonlinear Fractional Programming.” *Management Science* 13 (7): 492–98. <https://doi.org/10.1287/mnsc.13.7.492>.
- Duan, Ran, Hongxun Wu, and Renfei Zhou. 2023. “Faster Matrix Multiplication via Asymmetric Hashing.” In *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*, 2129–38. IEEE. <https://doi.org/10.1109/FOCS57990.2023.00130>.
- Dupont, Emilien, Marvin Eisenberger, Borislav Kozlovskii, Abbas Mehrabian, Francisco J. R. Ruiz, Abigail See, Renfei Zhou, Josh Alman, Virginia Vassilevska Williams, and Matej Balog. 2026. “Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve.” <https://arxiv.org/abs/2608.16884>.
- Gill, Philip E., Walter Murray, and Michael A. Saunders. 2002. “SNOPT: An SQP Algorithm for Large-Scale Constrained Optimization.” *SIAM Journal on Optimization* 12 (4): 979–1006. <https://doi.org/10.1137/S1052623499350013>.
- Kingma, Diederik P., and Jimmy Ba. 2015. “Adam: A Method for Stochastic Optimization.” In *3rd International Conference on Learning Representations (ICLR 2015)*. San Diego. <https://arxiv.org/abs/1412.6980>.
- Le Gall, François. 2014. “Powers of Tensors and Fast Matrix Multiplication.” In *Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation (ISSAC 2014)*, 296–303. ACM. <https://doi.org/10.1145/2608628.2608664>.
- Lemaréchal, Claude, François Oustry, and Claudia Sagastizábal. 2000. “The $\mathcal{U}$ -Lagrangian of a Convex Function.” *Transactions of the American Mathematical Society* 352 (2): 711–29. <https://doi.org/10.1090/S0002-9947-99-02243-6>.

- Lewis, Adrian S. 2002. “Active Sets, Nonsmoothness, and Sensitivity.” *SIAM Journal on Optimization* 13 (3): 702–25. <https://doi.org/10.1137/S1052623401387623>.
- Mifflin, Robert, and Claudia Sagastizábal. 2005. “A $\mathcal{VU}$ -Algorithm for Convex Minimization.” *Mathematical Programming* 104 (2–3): 583–608. <https://doi.org/10.1007/s10107-005-0630-3>.
- Moura, Leonardo de, and Sebastian Ullrich. 2021. “The Lean 4 Theorem Prover and Programming Language.” In *Automated Deduction – CADE 28*, 625–35. https://doi.org/10.1007/978-3-030-79876-5_37.
- Nocedal, Jorge, and Stephen J. Wright. 2006. *Numerical Optimization*. Second. Springer Series in Operations Research and Financial Engineering. New York: Springer. <https://doi.org/10.1007/978-0-387-40065-5>.
- Sinkhorn, Richard, and Paul Knopp. 1967. “Concerning Nonnegative Matrices and Doubly Stochastic Matrices.” *Pacific Journal of Mathematics* 21 (2): 343–48. <https://doi.org/10.2140/pjm.1967.21.343>.
- Steihaug, Trond. 1983. “The Conjugate Gradient Method and Trust Regions in Large Scale Optimization.” *SIAM Journal on Numerical Analysis* 20 (3): 626–37. <https://doi.org/10.1137/0720042>.
- Vassilevska Williams, Virginia, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. 2024. “New Bounds for Matrix Multiplication: From Alpha to Omega.” In *Proceedings of the 2024 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 3792–3835. SIAM. <https://doi.org/10.1137/1.9781611977912.134>.

Appendix A. Artifact identities

The raw generated Lean module contains 459,016,569 bytes and has SHA-256

```
7a2b439bed7c9caf3e3e59a0ee338b84c21efa51acacda2419448465d83f2786
```

Its lossless compressed payload is `Omega_1a05ebc4e32c1a21.lean.gz`, with 85,386,874 bytes. The package inventory binds both compressed and raw identities. The original full normative source manifest contains 1,259 files and has SHA-256

```
b56790ee4a5c9701f7f2f0754b870a98883fec474536ec3d23311e6b2750deed
```

This original manifest is distinct from the reduced reader workspace. The exact claim, canonical certificate, module and assurance are matched by the supplied inventory.

Appendix B. Aggregate data

The paper supplies three small tables in `data/`:

- `search-progress.csv`, which retains the rejected stage and its preceding incumbent;
- `width-efficiency.csv`, which also keeps the two unavailable comparisons;
- `model-predictions.csv`.

Comparison labels A–H follow the order of the eight available marginal funding comparisons; G is the eleven-direction case.

Before preparing the paper, the 140 consumed source-metadata bindings for these tables were rehashed. That check concerns the plotted observations; the large numerical payloads and full mathematical assurance closure were not re-run for the charts. `data/manifest.json` binds the supplied aggregate tables, figures and the endpoint-summary values. It contains no private filesystem paths or numerical checkpoint archives.

Appendix C. Clean-room verification record

The record comes from the release assets of `14-2026-10-04-search043`, run in an empty directory on a MacBook Pro (MacBookPro18,4, Apple M1 Max, 64 GiB), macOS 27.0, with `CARGO_BUILD_JOBS=1` and `LEAN_NUM_THREADS=1`. Other workloads shared the machine throughout, so elapsed times are upper bounds for this hardware.

Table 6: Clean-room reader run.

Step	Command	Result	Elapsed
Download and checksums	gh release download + release-assets- SHA256SUMS.txt	pass (3/3 checksums)	329 s
File identities	python3 verify-files.py	pass (135 files)	2 s
Rust build	cargo build --release --locked -p mm-cli --bin mm	pass	59 s
Lean build	lake exe cache get; lake --no-cache build MatrixMath	pass (2,229 jobs, 0 errors)	275 s + 91 s
Exact Rust check	mm verify ... --skip-lean --json	VERIFIED, output byte-identical	856 s
Full CN certification	mm prove ... --profile cn --json	exit 0	16,209 s (4.5 h)
Module digest comparison	regenerated vs manifest.json	byte-identical	—
Compiled assurance comparison	statements, digests, axioms vs compiled- assurance.json	byte-identical	—

- **Peak whole-system memory in use during CN:** 40.0 GiB, including other workloads; the peak resident set of the mm, lake and lean process tree was 15.4 GiB
- **Toolchains:** Rust 1.94.0 and Lean 4.33.0 (commit d8b18978), both from the package’s toolchain pins; Cargo.lock and lake-manifest.json were unchanged after dependency fetch.
- **New reduced source-closure digest:** 101e0ae6b87074d8acee13c21446376a17b9657941b759752be9c1b1299b7029, recomputed independently over the 22 shipped .lean files.
- **Receipt:** RECEIPT.md, SHA-256 68cbf93fea01cb5611d6ed1012fbf671402ec39aab7510f229690d163b9c4fcb.

Appendix D. Structure of the search: finite-step laws and the optimum

The following are measurements from the saved search records, not new claims about the bound. The records cover:

- 412 trial evaluations from the continuation;
- 49 model origins across the exploratory and continuation phases;
- the search043 model origin.

The records are sequential and adaptive, so counts are records, not independent experiments.

Finite-step laws. These support §5.3 and §6.2.

Table 7: Finite-step regularities of tangent search with restoration.

Observation	Value
Slack growth, full step / half step (exact quadratic: 4)	median 3.99, range 3.72–4.13 ($n = 53$ matched pairs)
Quadratic prediction vs. raw smooth-mixture gain	agreement within 1% ($n = 20$); uncapped full/half gain ratio exactly 4/3
Restored hard gain / raw mixture gain	median 1.000; only the two correction-limited wide steps fall below (0.79, 0.90)
Size of the restoration step relative to the accepted step	at most 0.9%

Two consequences follow:

- **The nonsmoothness is fully repairable.** Raw proposals lose 10–150× their smooth gain to slack, yet restoration returns essentially all of the smooth gain.
- **The limits are elsewhere.** Progress is limited by the small remaining projected gradient and by ill-conditioned reduced curvature, not by nonsmoothness.

Branch multipliers.

- All 2,940 fitted branch weights across 49 origins lie in $[0.3273, 0.3393]$.
- Small residuals in the least-squares fit (6) leave an identifiability band around each weight. In 88–100% of origins, 1/3 lies inside that band at every site except the level-three interior sites.
- At the level-three interior sites, departures of $2\text{--}3 \times 10^{-3}$ are resolved, about 1.5–2× the band.
- Forcing all weights to 1/3 raises the stationarity residual tenfold. Freeing the level-three sites alone reduces this to 2.4× the fitted residual.
- So the near-uniformity is largely real but not exact. We know of no theorem that forces it.

Late-search dynamics.

- Late accepted steps were often nearly collinear: the cosine between consecutive steps was at least 0.96 in two long runs. The binding coordinate of the 0.25 displacement cap was frequently a near-zero-weight root logit.
- Three of the six root regions drained monotonically toward zero over the campaign (weights $0.016 \rightarrow 1.6 \times 10^{-6}$, $0.040 \rightarrow 1.9 \times 10^{-4}$ and $0.0125 \rightarrow 3.4 \times 10^{-6}$), with positive reduced costs.
- The surviving regions place a different coordinate in the distinguished (penalized) role, one region per coordinate.
- This draining explains only about 2% of the late gain, to first order.

Structure of the optimum.

- *Root types.* For the surviving regions, the root shape distributions are nearly invariant under coordinate permutations: the maximum L_1 distance to any permuted copy is at most 7×10^{-4} , and the regions nearly share one distribution ($L_1 \approx 3 \times 10^{-4}$).
- *Node masses.* These are very sparse:

Level	Nodes with mass $< 10^{-12}$	Entropy-effective support
2	80.5%	about 3,500 of 1.49 million nodes
3	46%	about 920 of 64,260
4	10%	about 100 of 918

Appendix E. Where the remaining slack is, and what cannot reach it

All quantities are at the search043 model origin (the accepted stage-042 endpoint).

Loss budget. Each min-of-three candidate splits exactly into entropy gains minus losses: the max-entropy penalty P on the distinguished coordinate, and the conditional-entropy terms η on the others. The split reproduces all 60 candidates to 10^{-15} . First-order ceilings use the fitted branch weights as shadow prices (envelope theorem; the contrast Jacobian has rank 40).

Table 9: First-order value of removing each loss.

Loss	First-order ω ceiling if removed
Max-entropy penalty P (root, level 4, level 3)	$\leq 1.6 \times 10^{-6}$ in total
η conditioning gap, level 4	4.5×10^{-4}
η conditioning gap, root	2.0×10^{-4}
η conditioning gap, level 3	7.6×10^{-5}

The conditioning gap is the entropy of each group’s mixture minus the shapes’ own entropies. It is nonnegative by concavity.

Re-optimized relaxation (not a bound). Replacing η ’s grouped mixture by per-shape entropies and re-optimizing measures what a perfect conditioning mechanism would be worth. Both arms of each comparison had the same start and budget.

- **Level three, 50,000 Adam steps:** the gap between the idealized arm and the control is 2.4×10^{-3} . About 55% comes from the root terms.
- **Level four, 300 steps:** at least 8.6×10^{-5} , while the control did not improve at all.
- **Caveat:** the level-three runs started from an older symmetric point. Their control plateaued above the certified level-three value of (BSD 2026).

Admissibility.

- Information is admissible for compatibility only if it is fixed by the level- ℓ Y-block or pinned by the X-block (cf. Alman et al., Claim 5.7).
- *Parity.* Parity of a second coordinate would carry about half of the first-order gap, but it is neither, so it is not admissible.
- *Grade-1 constraint.* Duan, Wu and Zhou rejected an exact grade-1 constraint (Duan, Wu, and Zhou 2023). A set-valued version that counts the free position as slack recovers nothing: the freedom costs 1–2 bits per position, about 400–1,000 $\times$ the information gained.
- *Small cases.* At the second power, brute-force enumeration confirms Claim 5.7 and finds that 6–10% of containing triples are not fully shape-compatible. So “containment implies full-shape compatibility” is false.
- *Containment ceiling.* The best admissible definition, “compatible = containable”, recovers:
 - 0 at level three (exact: the current optimum is containment-feasible in all 1,127,520 groups);
 - at most 6.9×10^{-10} at level four (36,504 groups);
 - at most 1.4×10^{-9} at the root (186 groups; full-shape gap 2.0×10^{-4}).

Each upper bound comes from an explicit coupling that satisfies the containment constraints, verified independently with marginal residuals below 10^{-10} ; the lower bound is the trivial 0. Feasibility verdicts from a linear-programming solver at default tolerances were unreliable near the boundary and were not used. All values hold at the search043 optimum only.